

AIAC_ASSIGNMENT_5.1 AND 6

NAME : E.ANJANI

BATCH : 32

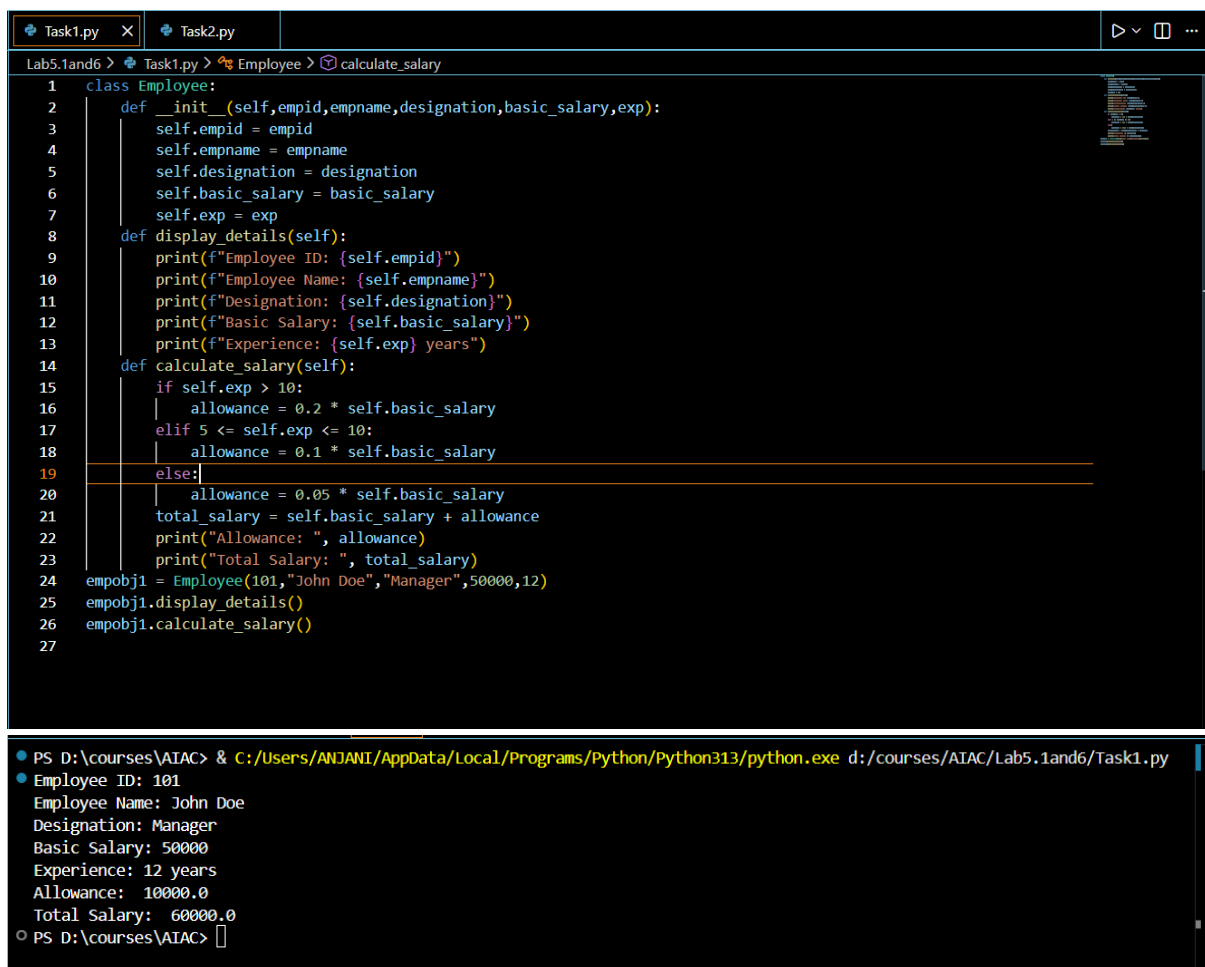
HNO:2303A52419

Task 1:

Employee Data: Create Python code that defines a class named `Employee` with the following attributes: `empid`, `empname`, `designation`, `basic\_salary`, and `exp`. Implement a method `display\_details()` to print all employee details. Implement another method `calculate\_allowance()` to determine additional allowance based on experience:

- If `exp > 10 years` → allowance = 20% of `basic\_salary`
- If $5 \leq \text{exp} \leq 10 \text{ years}$ → allowance = 10% of `basic\_salary`
- If `exp < 5 years` → allowance = 5% of `basic\_salary`

Finally, create at least one instance of the `Employee` class, call the `display\_details()` method, and print the calculated allowance.



```
Task1.py x Task2.py
Lab5.1and6 > Task1.py > Employee > calculate_salary
1 class Employee:
2     def __init__(self,empid,empname,designation,basic_salary,exp):
3         self.empid = empid
4         self.empname = empname
5         self.designation = designation
6         self.basic_salary = basic_salary
7         self.exp = exp
8     def display_details(self):
9         print(f"Employee ID: {self.empid}")
10        print(f"Employee Name: {self.empname}")
11        print(f"Designation: {self.designation}")
12        print(f"Basic Salary: {self.basic_salary}")
13        print(f"Experience: {self.exp} years")
14    def calculate_salary(self):
15        if self.exp > 10:
16            allowance = 0.2 * self.basic_salary
17        elif 5 <= self.exp <= 10:
18            allowance = 0.1 * self.basic_salary
19        else:
20            allowance = 0.05 * self.basic_salary
21        total_salary = self.basic_salary + allowance
22        print("Allowance: ", allowance)
23        print("Total Salary: ", total_salary)
24    empobj1 = Employee(101,"John Doe","Manager",50000,12)
25    empobj1.display_details()
26    empobj1.calculate_salary()
27

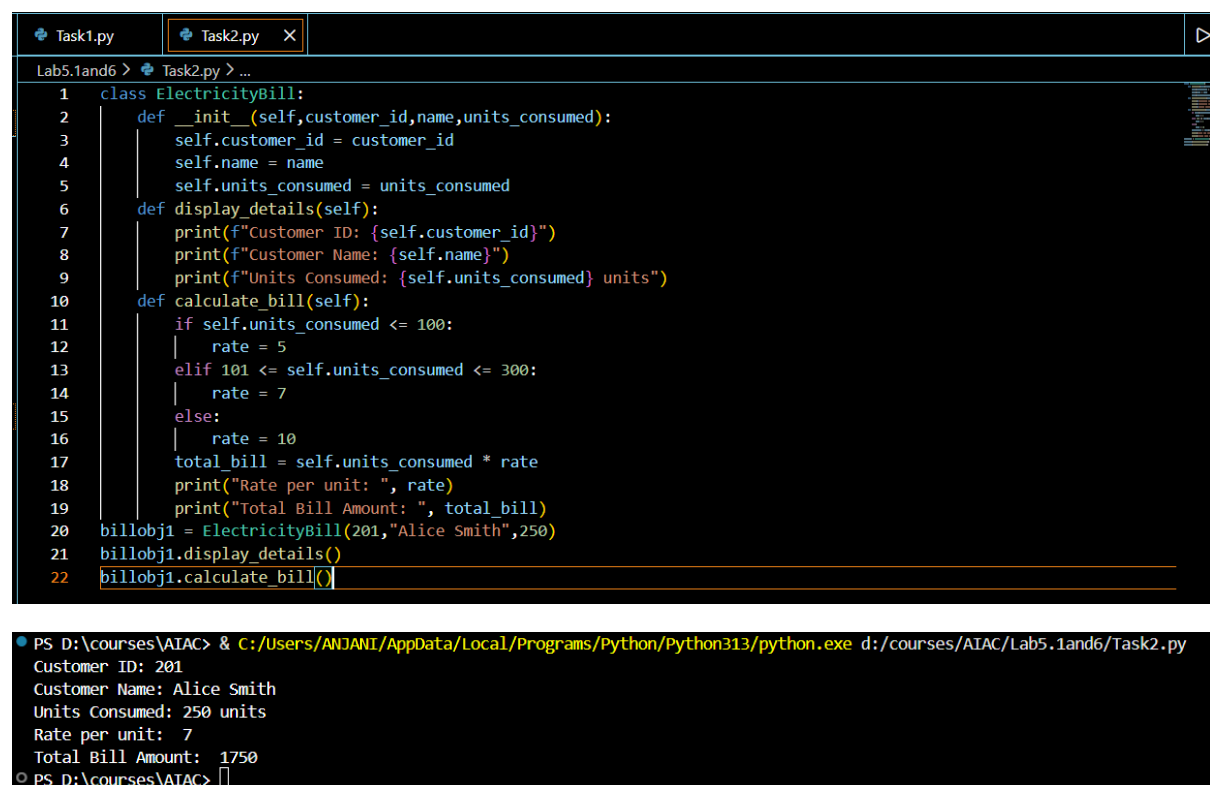
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab5.1and6/Task1.py
Employee ID: 101
Employee Name: John Doe
Designation: Manager
Basic Salary: 50000
Experience: 12 years
Allowance: 10000.0
Total Salary: 60000.0
PS D:\courses\AIAC>
```

Task 2:

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where:

- Units $\leq$ 100 $\rightarrow$ ₹5 per unit
- 101 to 300 units $\rightarrow$ ₹7 per unit
- More than 300 units $\rightarrow$ ₹10 per unit

Create a bill object, display details, and print the total bill amount.



```
Task2.py
Lab5.1and6 > Task2.py > ...
1 class ElectricityBill:
2     def __init__(self, customer_id, name, units_consumed):
3         self.customer_id = customer_id
4         self.name = name
5         self.units_consumed = units_consumed
6     def display_details(self):
7         print(f"Customer ID: {self.customer_id}")
8         print(f"Customer Name: {self.name}")
9         print(f"Units Consumed: {self.units_consumed} units")
10    def calculate_bill(self):
11        if self.units_consumed <= 100:
12            rate = 5
13        elif 101 <= self.units_consumed <= 300:
14            rate = 7
15        else:
16            rate = 10
17        total_bill = self.units_consumed * rate
18        print("Rate per unit: ", rate)
19        print("Total Bill Amount: ", total_bill)
20    billobj1 = ElectricityBill(201, "Alice Smith", 250)
21    billobj1.display_details()
22    billobj1.calculate_bill()
```

```
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab5.1and6/Task2.py
Customer ID: 201
Customer Name: Alice Smith
Units Consumed: 250 units
Rate per unit: 7
Total Bill Amount: 1750
PS D:\courses\AIAC>
```

Task 3:

Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product\_id`, `product\_name`, `price`, and `category`. Implement a method `display\_details()` to print product details. Implement another method `calculate\_discount()` where:

- Electronics $\rightarrow$ 10% discount
- Clothing $\rightarrow$ 15% discount
- Grocery $\rightarrow$ 5% discount

Create at least one product object, display details, and print the final price after discount.

```
Task1.py Task2.py Task3.py X
Lab5.1and6 > Task3.py > ...
1 class product:
2     def __init__(self,product_id,product_name,price,category):
3         self.product_id = product_id
4         self.product_name = product_name
5         self.price = price
6         self.category = category
7     def display_details(self):
8         print(f"Product ID: {self.product_id}")
9         print(f"Product Name: {self.product_name}")
10        print(f"Price: {self.price}")
11        print(f"Category: {self.category}")
12    def calculate_discount(self):
13        if self.category.lower() == "Electronics":
14            | discount = 0.1 * self.price
15        elif self.category.lower() == "Clothing":
16            | discount = 0.15 * self.price
17        else:
18            | discount = 0.05 * self.price
19        discounted_price = self.price - discount
20        print("Discount: ", discount)
21        print("Price after Discount: ", discounted_price)
22    prodobj1 = product(301,"Smartphone",20000,"Electronics")
23    prodobj1.display_details()
24    prodobj1.calculate_discount()
25
PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab5.1and6/Task3.py
Product ID: 301
Product Name: Smartphone
Price: 20000
Category: Electronics
Discount: 1000.0
Price after Discount: 19000.0
PS D:\courses\AIAC> 
```

Task 4:

Book Late Fee Calculation- Create Python code that defines a class named `LibraryBook` with attributes: `book\_id`, `title`, `author`, `borrower`, and `days\_late`. Implement a method `display\_details()` to print book details, and a method `calculate\_late\_fee()` where:

- Days late $\leq 5 \rightarrow ₹5$ per day
- 6 to 10 days late $\rightarrow ₹7$ per day
- More than 10 days late $\rightarrow ₹10$ per day

Create a book object, display details, and print the late fee.

```
Task1.py Task2.py Task3.py Task4.py
Lab5.1and6 > Task4.py > ...
1 class LibraryBook:
2     def __init__(self,book_id, title, author, borrower, days_late):
3         self.book_id = book_id
4         self.title = title
5         self.author = author
6         self.borrower = borrower
7         self.days_late = days_late
8     def display_details(self):
9         print(f"Book ID: {self.book_id}")
10        print(f"Title: {self.title}")
11        print(f"Author: {self.author}")
12        print(f"Borrower: {self.borrower}")
13        print(f"Days Late: {self.days_late} days")
14    def calculate_late_fee(self):
15        if self.days_late <= 5:
16            fee_per_day = 1
17        elif 6 <= self.days_late <= 10:
18            fee_per_day = 7
19        else:
20            fee_per_day = 10
21        total_fee = self.days_late * fee_per_day
22        print("Late Fee per day: ", fee_per_day)
23        print("Total Late Fee: ", total_fee)
24    bookobj1 = LibraryBook(401, "The Great Gatsby", "F. Scott Fitzgerald", "Bob Johnson", 8)
25    bookobj1.display_details()
26    bookobj1.calculate_late_fee()

PS D:\courses\AIAC> & C:/Users/ANJANI/AppData/Local/Programs/Python/Python313/python.exe d:/courses/AIAC/Lab5.1and6/Task4.py
Book ID: 401
Title: The Great Gatsby
Author: F. Scott Fitzgerald
Borrower: Bob Johnson
Days Late: 8 days
Late Fee per day: 7
Total Late Fee: 56
PS D:\courses\AIAC>
```

Task 5:

Student Performance Report - Define a function `student_report(student_data)` that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student
- Determine pass/fail status (pass ≥ 40)
- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output.

```
Lab5.1and6 > Task5.py > ...
1 def student_report(student_data):
2     report = {}
3     for student in student_data:
4         name = student.get('name', 'Unknown')
5         grades = student.get('grades', {})
6         if grades:
7             average_grade = sum(grades) / len(grades)
8             status = 'Pass' if average_grade >= 40 else 'Fail'
9         else:
10            average_grade = 0
11            status = 'Fail'
12            report[name] = {
13                'average_grade': average_grade,
14                'status': status
15            }
16    return report
17 students = [
18     {'name': 'Ana', 'grades': [85, 78, 92]},
19     {'name': 'Janu', 'grades': [45, 38, 50]},
20     {'name': 'Hima', 'grades': [30, 25, 40]},
21 ]
22 report = student_report(students)
23 print(report)
24
```

```
PS D:\courses\AIAC> & C:\Users\ANJANI\AppData\Local\Programs\Python\Python313\python.exe d:/courses/AIAC/Lab5.1and6/Task5.py
{'Ana': {'average_grade': 85.0, 'status': 'Pass'}, 'Janu': {'average_grade': 44.333333333333336, 'status': 'Pass'}, 'Hima': {'average_grade': 31.666666666666668, 'status': 'Fail'}}
PS D:\courses\AIAC>
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

```
Task1.py Task2.py Task3.py Task4.py Task5.py Task6.py Task9.py
Lab5.1and6 > Task6.py > TaxiRide > calculate_fare
1 class TaxiRide:
2     def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):
3         self.ride_id = ride_id
4         self.driver_name = driver_name
5         self.distance_km = distance_km
6         self.waiting_time_min = waiting_time_min
7
8     def display_details(self):
9         print("Ride ID:", self.ride_id)
10        print("Driver Name:", self.driver_name)
11        print("Distance:", self.distance_km, "km")
12        print("Waiting Time:", self.waiting_time_min, "minutes")
13
14    def calculate_fare(self):
15        fare = 0
16        if self.distance_km <= 10:
17            fare = self.distance_km * 15
18        elif self.distance_km <= 30:
19            fare = (10 * 15) + ((self.distance_km - 10) * 12)
20        else:
21            fare = (10 * 15) + (20 * 12) + ((self.distance_km - 30) * 10)
22
23        waiting_charge = self.waiting_time_min * 2
24        total_fare = fare + waiting_charge
25
26        print("Travel Fare:", fare)
27        print("Waiting Charge:", waiting_charge)
28        print("Total Fare:", total_fare)
29
30 rideobj1 = TaxiRide(501, "David Lee", 15, 10)
31 rideobj1.display_details()
32 rideobj1.calculate_fare()
```

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task6.py
● Ride ID: 501
  Driver Name: David Lee
  Distance: 15 km
  Waiting Time: 10 minutes
  Travel Fare: 210
  Waiting Charge: 20
  Total Fare: 230
○ PS D:\courses\AIAC>
```

Ln 22, Col 1 S

Task 7:

Statistics Subject Performance - Create a Python function `statistics_subject(scores_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

```
Lab5.1and6 > Task7.py > ...
1 def statistics_subject(scores_list):
2     if not scores_list:
3         return None, None, None, None, None
4
5     highest_score = max(scores_list)
6     lowest_score = min(scores_list)
7     average_score = sum(scores_list) / len(scores_list)
8
9     passed_students = sum(1 for score in scores_list if score >= 40)
10    failed_students = len(scores_list) - passed_students
11
12    return highest_score, lowest_score, average_score, passed_students, failed_students
13
14
15 # Example usage
16 scores = [85, 42, 78, 90, 33, 67, 49, 55, 29, 100]
17
18 high, low, avg, passed, failed = statistics_subject(scores)
19
20 print("Highest Score:", high)
21 print("Lowest Score:", low)
22 print("Average Score:", avg)
23 print("Number of Students Passed:", passed)
24 print("Number of Students Failed:", failed)
25
```

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task7.py
Highest Score: 100
Lowest Score: 29
Average Score: 62.8
Number of Students Passed: 8
Number of Students Failed: 2
PS D:\courses\AIAC>
```

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

- Code for both methods.
- Transparent explanation of time complexity.
- Comparison highlighting efficiency improvements.

```
Lab5.1and6 > Task8.py > ...
1 #generate a well commented code to check if a number is prime
2 def is_prime(num):
3     """
4     This function checks if a given number is prime.
5
6     Parameters:
7     num (int): The number to be checked for primality.
8
9     Returns:
10    bool: True if the number is prime, False otherwise.
11    """
12    # Check if the number is less than 2, which are not prime numbers
13    if num < 2:
14        return False
15    # Check for factors from 2 to the square root of the number
16    for i in range(2, int(num**0.5) + 1):
17        # If num is divisible by any number other than 1 and itself, it's not prime
18        if num % i == 0:
19            return False
20    # If no factors were found, the number is prime
21    return True
22 # Example usage:
23 number = 29
24 if is_prime(number):
25     print(f"{number} is a prime number.")
26 else:
27     print(f"{number} is not a prime number.")
28
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task8.py
29 is a prime number.
PS D:\courses\AIAC>
```

```
Lab5.1and6 > Task8.py > ...
29 #generate a well commented and optimized code to check if a number is prime
30 def is_prime_optimized(num):
31     """
32     This function checks if a given number is prime using an optimized approach.
33
34     Parameters:
35     num (int): The number to be checked for primality.
36
37     Returns:
38     bool: True if the number is prime, False otherwise.
39     """
40    # Check if the number is less than 2, which are not prime numbers
41    if num < 2:
42        return False
43    # Check for 2 and 3 explicitly
44    if num in (2, 3):
45        return True
46    # Eliminate even numbers greater than 2 and multiples of 3
47    if num % 2 == 0 or num % 3 == 0:
48        return False
49    # Check for factors from 5 to the square root of the number, skipping even numbers
50    for i in range(5, int(num**0.5) + 1, 6):
51        if num % i == 0 or num % (i + 2) == 0:
52            return False
53    # If no factors were found, the number is prime
54    return True
55 # Example usage:
56 number = 456
57 if is_prime_optimized(number):
58     print(f"{number} is a prime number.")
59 else:
60     print(f"{number} is not a prime number.")
61
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task8.py
456 is not a prime number.
PS D:\courses\AIAC>
```

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.
2. Ask AI to explain base cases and recursive calls.

Expected Output:

- Well-commented recursive code.
- Clear explanation of how recursion works.
- Verification that explanation matches actual execution.

```
Lab5.1and6 > Task9.py > ...
1  #generate well commented code for fibonacci series using recursion
2  def fibonacci(n):
3      """
4      This function returns the nth Fibonacci number using recursion.
5
6      Parameters:
7      n (int): The position in the Fibonacci series to retrieve.
8
9      Returns:
10     int: The nth Fibonacci number.
11     """
12     # Base case: the first two Fibonacci numbers are 0 and 1
13     if n < 0:
14         raise ValueError("Input should be a non-negative integer.")
15     elif n == 0:
16         return 0
17     elif n == 1:
18         return 1
19     else:
20         # Recursive case: sum of the two preceding Fibonacci numbers
21         return fibonacci(n - 1) + fibonacci(n - 2)
22 # Example usage:
23 position = 10
24 print(f"The {position}th Fibonacci number is: {fibonacci(position)}")

PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task9.py
The 10th Fibonacci number is: 55
PS D:\courses\AIAC>
```

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.

- Validation that explanations align with runtime behavior.

```

Lab5.1and6 > Task10.py > read_and_process_file
1 #Generate a well commented code with proper error handling and clear explanations for each exception.
2 def read_and_process_file(filename):
3     """
4     This function reads numbers from a file and calculates their average.
5     Each number should be on a separate line.
6     """
7     try:
8         # Attempt to open and read the file
9         file = open(filename, "r")
10        lines = file.readlines()
11        file.close()
12        numbers = []
13        # Convert each line into an integer
14        for line in lines:
15            number = int(line.strip()) # May raise ValueError
16            numbers.append(number)
17        # Check if file is empty
18        if len(numbers) == 0:
19            raise ValueError("File is empty. No data to process.")
20        # Calculate average
21        average = sum(numbers) / len(numbers)
22        print("Numbers in file:", numbers)
23        print("Average:", average)
24    except FileNotFoundError:
25        # Happens if the file does not exist
26        print("Error: File not found. Please check the file name.")
27    except ValueError as ve:
28        # Happens if conversion to int fails OR file is empty
29        print("Error:", ve)
30    except Exception as e:
31        # Catches any unexpected error
32        print("Unexpected error occurred:", e)
33    finally:
34        # Always runs whether error occurs or not
35        print("File processing completed.")
36
37 # Example usage
38 filename = "data.txt"
39 read_and_process_file(filename)

```

```

PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab5.1and6/Task10.py
Error: File not found. Please check the file name.
File processing completed.
PS D:\courses\AIAC>

```